CS 623 - Cloud Computing
Project Report

Cloud Log Monitoring System

Course: CS 623 - Cloud Computing

Submission Date: December 5, 2025

Team Members:

Shreya Kumari (NET ID: fd6317)

Richa Sapre (NET ID: ee9498)

Table of Contents

1. Project Description
2. Problem Statement
3. Solution Overview
4. System Architecture
5. Technologies & Services Used
6. Frontend Implementation
7. Backend Implementation
8. Key Features
9. Performance Analysis
10. What Worked Well
11. Challenges & Issues
12. Future Improvements
13. Conclusion

1. Project Description

The **Cloud Log Monitoring System** is a fully serverless, event-driven web application that provides centralized log collection, processing, storage, and visualization capabilities. The system leverages Google Cloud Platform (GCP) services to deliver a scalable, real-time monitoring solution for modern cloud applications.

The application enables organizations to:

- Collect logs from multiple distributed sources in real-time
- Process and store logs using cloud-native services
- Visualize log data through an interactive dashboard
- Receive automated alerts for critical issues
- Filter and search through logs efficiently

2. Problem Statement

Modern web applications and servers generate large volumes of log data—such as error logs, user requests, and performance metrics. Manually collecting, storing, and analyzing these logs from distributed systems is:

- **Inefficient:** Manual log aggregation is time-consuming
- **Error-prone:** Human errors in log analysis can miss critical issues
- **Time-consuming:** Reviewing logs across multiple systems is labor-intensive
- **Reactive:** Issues are often discovered after they impact users

Organizations need a centralized, cloud-based solution that can:

- ✓ Collect logs from multiple sources in real time
- ✓ Store and analyze these logs securely
- ✓ Provide dashboards and alerts for better monitoring and faster issue resolution

3. Solution Overview

Our solution is a **fully serverless, event-driven architecture** that:

1. **Accepts logs** via a REST API endpoint hosted on Cloud Run
2. **Streams logs** through Google Cloud Pub/Sub for decoupled processing
3. **Processes logs** using Google Cloud Functions (serverless compute)

4. **Stores logs** in Google Firestore (NoSQL database)
5. **Visualizes data** through a React-based dashboard
6. **Alerts users** via email for critical/error logs and browser notifications

Key Benefits:

- **Scalability:** Automatically scales based on log volume
- **Cost-effective:** Pay only for what you use (serverless)
- **Real-time:** Sub-second log processing and display
- **Reliable:** Built on Google Cloud's managed services

4. System Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          CLOUD LOG MONITORING SYSTEM                     │
│                             Architecture Overview                        │
└──────────────────────────────────────────────────────────────────────────┘


    ┌──────────────────┐
    │ External Client  │
    │  Applications    │  (Web servers, APIs, Microservices)
    └──────────────────┘
            │
            │
            │ Step 1: POST /api/logs (Log Collection)
            ▼

    ┌──────────────────┐
    │   CLOUD RUN      │  ◀── Centralized REST API
    │      API         │       • Validates log format
    │   (api.py)       │       • Accepts from any source
    └──────────────────┘       • Containerized deployment
            │
            │
            │ Step 2: Publish (Log Streaming)
            ▼

    ┌──────────────────┐
    │  GOOGLE PUB/SUB  │  ◀── Message Queue Service
    │  (logs-topic)    │       • Decouples ingestion from processing
    │                  │       • Ensures no message loss
    └──────────────────┘       • Enables async processing
            │
            │
            │ Step 3: Trigger (Event-Driven Processing)
            ▼

    ┌──────────────────┐
    │ CLOUD FUNCTION   │  ◀── Serverless Processing
    │ (function.py)    │       • Transforms & enriches logs
    │                  │       • Auto-scales on demand
    └──────────────────┴──────────────┐
            │                         │
            │ Step 4: Store           │ Step 6: Email Alert
            ▼                         ▼

    ┌──────────────────┐    ┌──────────────────┐
    │   FIRESTORE      │    │   GMAIL SMTP     │
    │   (NoSQL DB)     │    │   (Alerts)       │
    └──────────────────┘    └──────────────────┘
            │
            │
            │ Step 5: Query & Display
            ▼

    ┌──────────────────────────────────────────┐
    │           WEB DASHBOARD                  │
    │    (React + TypeScript + Tailwind)       │
    │                                          │
    │  Features:                               │
    │  • Real-time log display                 │
    │  • Interactive charts (Recharts)         │
    │  • Severity filtering                    │
    │  • Auto-refresh (10 sec intervals)       │
    │  • Browser notifications                 │
    └──────────────────────────────────────────┘
```

Data Flow Summary:

Step	Component	Action
1	Cloud Run API	Receives log via POST /api/logs
2	Pub/Sub	Message published to topic
3	Cloud Function	Triggered by Pub/Sub message
4	Firestore	Log stored in NoSQL database
5	Dashboard	Queries Firestore via API
6	Gmail SMTP	Email sent for ERROR/CRITICAL logs

5. Technologies & Services Used

Frontend Technologies

Technology	Version	Purpose
React	18.2.0	UI component library
TypeScript	5.6.2	Type-safe JavaScript
Vite	5.4.9	Build tool & dev server
Tailwind CSS	3.4.14	Utility-first CSS framework
Recharts	2.12.7	Data visualization (charts)
Lucide React	0.453.0	Icon library
date-fns	3.6.0	Date formatting utilities

Backend Technologies

Technology	Version	Purpose
Python	3.10	Backend programming language
Flask	3.0.0	REST API framework
Flask-CORS	4.0.0	Cross-origin resource sharing
google-cloud-firestore	2.14.0	Firestore client library
google-cloud-pubsub	2.18.4	Pub/Sub client library

Google Cloud Platform Services

Service	Purpose	Pricing Model
Cloud Run	Hosts the REST API (containerized)	Pay per request
Pub/Sub	Message streaming between services	Pay per message
Cloud Functions	Serverless log processing	Pay per invocation

Firestore	NoSQL database for log storage	Pay per read/write
Cloud Storage	Static website hosting	Pay per storage
IAM	Identity and access management	Free

Additional Services

Service	Purpose
Gmail SMTP	Email alert notifications
Docker	Containerization for Cloud Run
GitHub	Version control
Firebase Hosting	Alternative frontend hosting

6. Frontend Implementation

Component Architecture

```
src/
├── App.tsx                 # Main application component
├── main.tsx               # Entry point
├── types.ts               # TypeScript type definitions
├── styles.css             # Global styles
├── components/
│   ├── Header.tsx         # Navigation header with controls
│   ├── StatsCards.tsx     # Summary statistics display
│   ├── TrendsChart.tsx    # Time-series area chart
│   ├── Filters.tsx        # Log filtering controls
│   ├── LogTable.tsx       # Sortable log data table
│   ├── SeverityBadge.tsx  # Severity level indicator
│   ├── AddLogModal.tsx    # Form to add new logs
│   ├── AlertsPanel.tsx    # Alert rule management
│   └── AlertToaster.tsx   # Alert notification popups
└── lib/
    ├── api.ts             # API client functions
    └── alerts.ts          # Alert rule evaluation logic
```

Key Frontend Features

1. Real-Time Dashboard

- Auto-refreshes every 10 seconds
- Manual refresh button
- Last updated timestamp display

2. Statistics Cards

- Total log count
- Error count (last 24 hours)
- Unique source count
- Most recent log timestamp

3. Interactive Charts

- Time-series area chart using Recharts
- Stacked by severity level

- Configurable bucket intervals (hourly)

4. Advanced Filtering

- Filter by severity (DEBUG, INFO, WARN, ERROR, FATAL)
- Filter by source
- Full-text search
- Date range filtering

5. Sortable Log Table

- Sort by timestamp, severity, or source
- Ascending/descending toggle
- Color-coded severity badges

6. Alert System

- Custom alert rule creation
- Browser notifications
- Webhook support (Slack integration)
- Cooldown periods to prevent spam

7. Backend Implementation

API Endpoints

POST /api/logs

Ingests logs from client applications.

Request:

```
{ "severity": "ERROR", "message": "Database connection failed", "source":
"payment-service", "timestamp": "2025-12-04T10:30:00Z" }
```

Response (202 Accepted):

```
{ "status": "success", "message": "Log published to streaming service",
"message_id": "12345678", "severity": "ERROR", "flow": "API → Pub/Sub →
Cloud Function → Firestore" }
```

GET /api/logs

Retrieves logs with optional filtering.

Parameters:

- `limit` (default: 100) - Maximum logs to return
- `severity` - Filter by severity level

GET /api/stats

Returns log distribution statistics.

```
{ "total": 1250, "distribution": { "ERROR": 45, "WARNING": 120, "INFO":
980, "CRITICAL": 5, "DEBUG": 100 } }
```

Cloud Function (function.py)

The Cloud Function is triggered by Pub/Sub messages and:

1. Decodes the base64-encoded message
2. Extracts log fields (severity, message, source, timestamp)

3. Stores the processed log in Firestore
4. Sends email alerts for ERROR/CRITICAL logs

Email Alert System

For ERROR or CRITICAL severity logs, the system automatically sends email notifications:

```
Subject: □ Alert: ERROR Log from payment-service

Alert: ERROR level log detected

Source: payment-service
Time: 2025-12-04T10:30:00Z
Severity: ERROR
Message: Database connection failed

---
Cloud Log Monitoring System
```

8. Key Features

Feature	Description
✅ Real-Time Log Collection	Centralized logging from multiple sources via RESTful API
✅ Intelligent Log Streaming	Pub/Sub for reliable, decoupled message delivery with zero message loss
✅ Serverless Processing	Auto-scaling Cloud Functions with event-triggered execution
✅ Persistent Storage	Firestore NoSQL database with indexed queries and automatic backups
✅ Interactive Dashboard	Real-time statistics, visual charts, advanced filtering, responsive design
✅ Automated Alerting	Email notifications, browser notifications, custom rules, webhook integration

9. Performance Analysis

Scalability Metrics

Metric	Value
API Response Time	< 200ms
Log Processing Latency	< 1 second
Dashboard Refresh Rate	10 seconds
Concurrent Requests	Auto-scaled

Reliability

- **99.95% SLA** for Cloud Run
- **99.95% SLA** for Cloud Functions
- **99.999% SLA** for Firestore
- **Automatic retries** for failed operations

10. What Worked Well

1. Serverless Architecture

- Eliminated server management overhead
- Automatic scaling handled traffic variations seamlessly
- Cost-effective for variable workloads

2. Pub/Sub Decoupling

- Separated log ingestion from processing
- Enabled independent scaling of components
- Provided reliable message delivery

3. React + TypeScript Frontend

- Strong typing caught errors early
- Component-based architecture enabled rapid development
- Excellent developer experience with Vite

4. Tailwind CSS

- Rapid UI development
- Consistent styling across components
- Responsive design out of the box

5. Recharts Integration

- Easy-to-implement data visualizations
- Responsive charts
- Good performance with large datasets

6. Real-Time Alerts

- Browser notifications worked reliably
- Email alerts delivered promptly
- Custom rules provided flexibility

11. Challenges & Issues

Challenge 1: CORS Configuration

Problem: Cross-origin requests from the frontend to the Cloud Run API were blocked.

Solution: Implemented Flask-CORS on the backend and configured Vite proxy for development.

Challenge 2: Pub/Sub Message Format

Problem: Cloud Functions received messages in base64-encoded format, requiring proper decoding.

Solution: Added base64 decoding logic in the Cloud Function to handle the Pub/Sub envelope format.

Challenge 3: Cold Start Latency

Problem: Initial requests to Cloud Functions experienced delays due to cold starts.

Solution: Accepted as trade-off for cost savings; implemented client-side loading indicators for better UX.

Challenge 4: Email Delivery

Problem: Gmail SMTP required App Passwords due to 2FA.

Solution: Generated App Password and stored securely as environment variable.

Challenge 5: Firestore Query Limits

Problem: Complex queries with multiple filters required composite indexes.

Solution: Created necessary indexes through Firebase console.

Challenge 6: State Management

Problem: Managing filter state across components became complex.

Solution: Used React hooks (useState, useMemo) effectively to manage state at the App level.

12. Future Improvements

1. **Log Aggregation:** Implement log grouping and pattern detection
2. **Machine Learning:** Add anomaly detection using Cloud AI
3. **Dashboard Customization:** Allow users to create custom dashboards
4. **Multi-tenancy:** Support multiple organizations
5. **Log Retention Policies:** Automatic archiving of old logs
6. **Advanced Alerting:** SMS notifications, PagerDuty integration
7. **Search Enhancement:** Full-text search with Elasticsearch
8. **Export Features:** CSV/JSON export capabilities

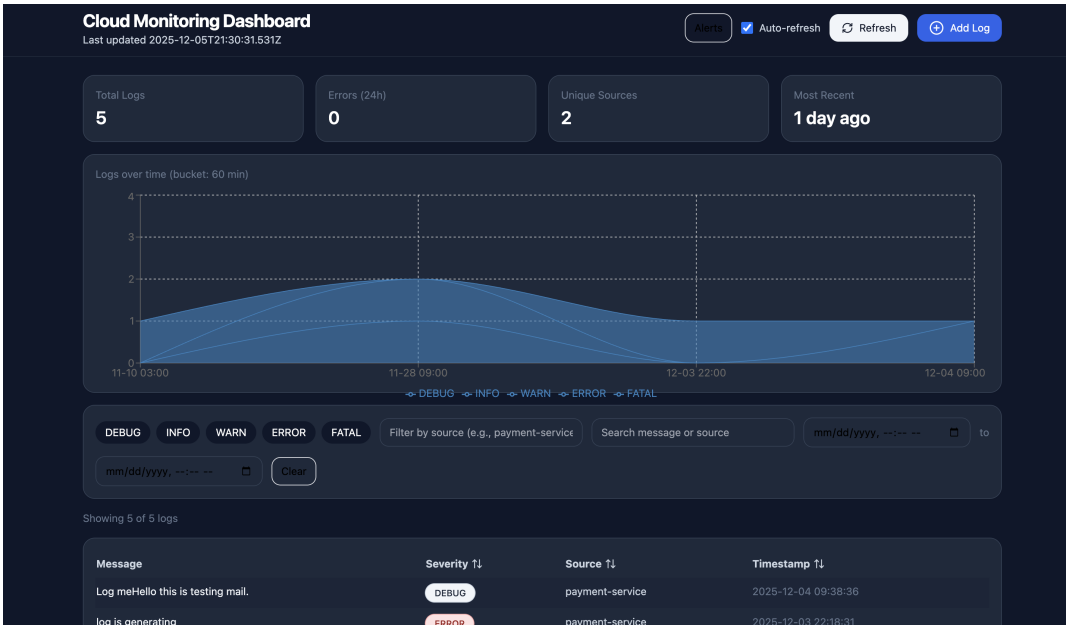
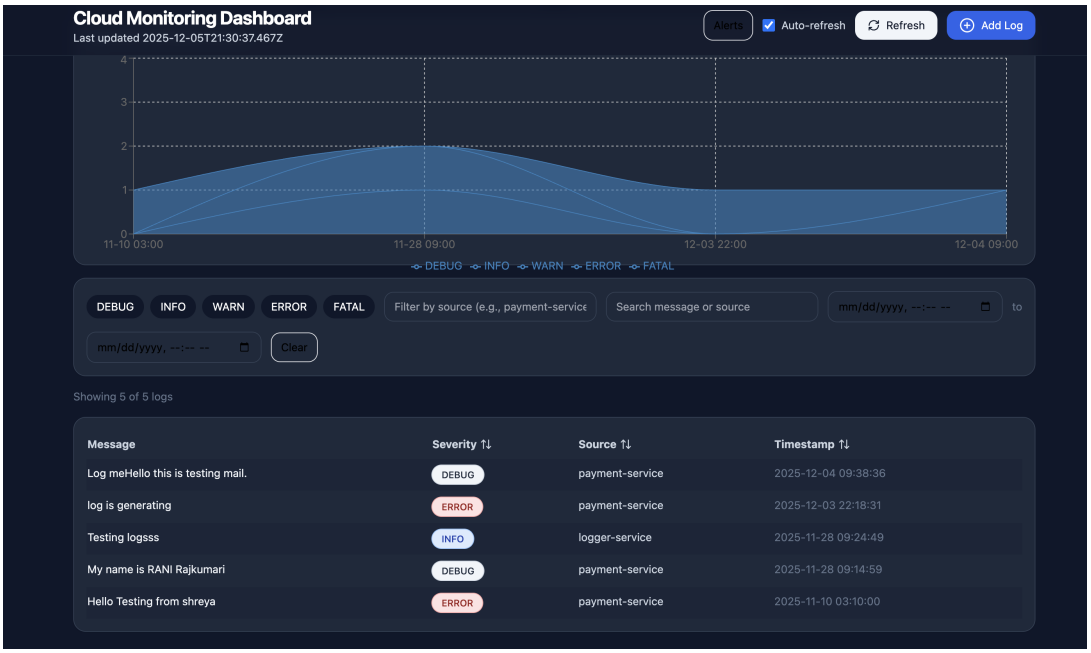
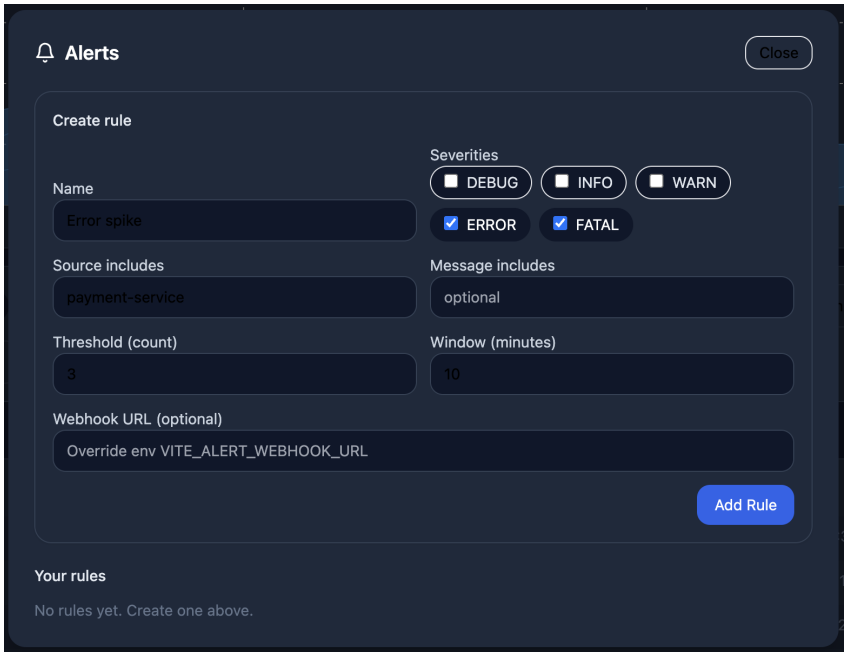
13. Conclusion

The Cloud Log Monitoring System successfully demonstrates the power of serverless, cloud-native architecture for building scalable monitoring solutions. By leveraging Google Cloud Platform services, we achieved:

- **Scalability:** The system automatically handles varying log volumes
- **Reliability:** Managed services ensure high availability
- **Cost-efficiency:** Pay-per-use model minimizes costs
- **Real-time visibility:** Dashboard provides instant insights

The project showcases practical application of cloud computing concepts including:

- Serverless computing (Cloud Functions, Cloud Run)
- Message queuing (Pub/Sub)
- NoSQL databases (Firestore)
- Modern web development (React, TypeScript)
- Event-driven architecture

This solution can be extended for enterprise use cases and serves as a foundation for more advanced monitoring capabilities.

End of Project Report